ROTEIRO DE LABORATÓRIO - POLLING E INTERRUPÇÕES COM ESP-IDF

Plataforma de Hardware para Internet das Coisas • Aula 10

PROBLEMA

Uma máquina precisa detectar quando uma peça toca rapidamente o sensor de fim de curso. Ao mesmo tempo, o ESP32 mantém um LED de estado piscando. O contato pode durar menos que o intervalo entre duas verificações do programa, mas nenhum evento pode ser perdido.

OBJETIVOS

• Comprovar experimentalmente a diferença entre polling e interrupção.

• Configurar uma interrupção de GPIO usando ESP-IDF.

• Manter a ISR curta e transferir o processamento para uma task.

• Usar fila, timestamp e debounce para registrar eventos válidos.

MATERIAIS

ESP32 DevKit clássico, botão ou chave fim de curso, LED, resistor de 330 Ω, protoboard, fios, cabo USB e computador com ESP-IDF instalado.

MONTAGEM

Ligue o botão ou fim de curso entre GPIO23 e GND. O programa ativará o pull-up interno. Ligue GPIO18 ao resistor de 330 Ω, o resistor ao ânodo do LED e o cátodo ao GND.



Um único circuito será usado em todas as etapas.

Solto, o GPIO23 lê `HIGH`. Acionado, lê `LOW`. O evento de interesse é a borda de descida.

PREPARAÇÃO

Abra o projeto `projeto_idf`, selecione a etapa em `main/main.c` e use:

```
idf.py set-target esp32
```

```
idf.py build
```

```
idf.py -p PORTA flash monitor
```

EXPERIMENTO 01 - POLLING: O PROGRAMA PERGUNTA

1. Ajuste `ETAPA` para `1`, compile e grave.
2. O LED muda de estado a cada 500 ms. A leitura do sensor ocorre no mesmo intervalo.
3. Faça dez acionamentos rápidos, tentando pressionar e soltar entre duas mudanças do LED.
4. Compare os acionamentos feitos com os registros exibidos no terminal.

Método	Acionamentos feitos	Eventos detectados	Eventos perdidos
Polling a cada 500 ms	10	_____	_____

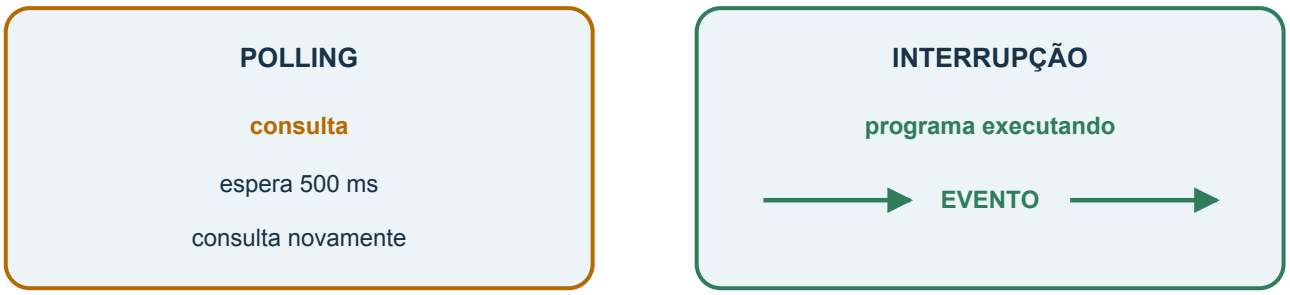
Questão 1. Por que um acionamento pode desaparecer completamente para o programa?

_____

EXPERIMENTO 02 - INTERRUPÇÃO: O HARDWARE AVISA

1. Ajuste `ETAPA` para `2`, compile e grave.
2. Repita dez acionamentos rápidos nas mesmas condições.
3. A ISR apenas incrementa um contador `volatile`; a mensagem continua sendo produzida no fluxo principal.

Método	Acionamentos feitos	Eventos detectados	Eventos perdidos
Interrupção por borda	10	_____	_____



Perguntar de tempos em tempos × ser avisado quando acontece

Questão 2. Complete com suas palavras:

• No polling, _____.

• Na interrupção, _____.

Questão 3. A interrupção torna o programa inteiro mais rápido ou garante atendimento quando o evento acontece? Justifique.

_____

EXPERIMENTO 03 - FILA E DEBOUNCE

O contador registra as ocorrências, mas não mostra quando elas aconteceram e o contato mecânico pode gerar repiques. Para examinar cada ocorrência e aplicar debounce, use uma fila.

1. Ajuste `ETAPA` para `3`.
2. A ISR cria um registro mínimo com `esp_timer_get_time()` e chama `xQueueSendFromISR()`.
3. A task recebe o registro, elimina repiques menores que 50 ms e imprime o evento.
4. Faça dez acionamentos em ritmos diferentes.



A ISR registra rapidamente; a task faz o trabalho demorado.

Acionamentos	Mensagens antes do debounce	Eventos válidos depois do debounce
10	_____	_____

Questão 4. Por que `ESP_LOGI()`, cálculos demorados e `vTaskDelay()` ficam fora da ISR?

_____

EXPERIMENTO 04 - MEDIR A DURAÇÃO DO EVENTO

1. Ajuste `ETAPA` para `4`. O GPIO será configurado com `GPIO_INTR_ANYEDGE`.
2. A borda de descida marca o início; a borda de subida marca o final.
3. Mantenha o botão pressionado por tempos diferentes e registre o resultado.

Teste	Duração indicada no terminal	Duração aproximada no cronômetro
Curto	_____ ms	_____ ms
Médio	_____ ms	_____ ms
Longo	_____ ms	_____ ms

Questão 5. Como essa técnica poderá medir o pulso `ECHO` do HC-SR04 na próxima prática?

_____

REGRAS DA ISR

• Fazer apenas o trabalho necessário para registrar ou sinalizar o evento.

• Usar funções próprias para contexto de interrupção, como `xQueueSendFromISR()`.

• Deixar logs, cálculos, acionamentos prolongados e esperas para uma task.

ENTREGA

Foto da montagem, tabelas preenchidas, cinco respostas e projeto ESP-IDF. Tempo previsto: **90 minutos**.

A diferença central deve aparecer nos dados: o polling depende do instante da consulta; a interrupção registra a ocorrência no instante do evento.